How to build your protection

A guide to `student_solution.py`: what is in it, how a request travels through it, where each kind of change goes, and how to test it. It shows you *where*. *What* to build is up to you.

Before you start

On the page, choose **6 · Student Solution** in the Scenario menu. Then open `student_solution.py` in your editor. From now on every button press on the page runs the code in that file.

1 The one file

Everything you write goes in `student_solution.py`. From top to bottom:

Part	What it is for
The comment at the top	The same guide as this page, in short, plus a **toolbox**: the Python functions already available in the file.
`SHARED_SECRET`	A value your client code and your car-server code both know. Change it.
`PROPERTIES`	The five badges on the page. Set the ones you believe your design provides.
`USER_ACTIONS`	Which actions the account may perform. One way to write the rule for `authorize`.
`PASSWORDS`	A place for whatever your car-server code needs to know about the password. Empty until you decide.
`create_request(...)` client	Builds one request when you press a button.
`verify_request(...)` car server	Looks at an arriving request and says yes or no.
`authorize(...)` car server	Says whether this user may do this action.
`maintenance_auth(...)` optional	The second door on the car. Write it only if you want to take that door over.

2 How a button press travels

You press ▲ The Client panel calls your `create_request`.

▼

create_request	Your code builds a request: a method (`POST`), a path (an address on the car), some headers (named values), and a body (text).
▼	
The network	The Interceptor can read the request here, change it, delay it, drop it, or send it again later.
▼	
verify_request	Your code looks at what arrived and returns `{"ok": True, ...}` or `{"ok": False, ...}`.
▼	
authorize	Your code says whether that user may do that action.
▼	
The car	Moves if both said yes. A refusal shows **rejected** in the logs; a crash in your code shows **error**.

Two dictionaries that keep values between presses

`client_state` is a dictionary (a set of named values) kept by the client. `server_state` is a dictionary kept by the car server. Both start empty, and anything you put in one is still there next time. They are two different dictionaries on two different sides: the car server cannot see `client_state`.

3 Your working loop

1. **Save** `student_solution.py`.
2. On the page, press **Reset Scenario**. This loads your saved file, empties both dictionaries, puts the car back in the middle, and clears the logs.
3. Press an arrow. Read the **Client request log** and the **Network log**. If your `verify_request` said no, the reason you wrote appears there.
4. Press **Run security checks** in the Car server panel. Each check starts with its own fresh copy of your saved file, so you can press it any time without Reset Scenario. Each line is one kind of attack. **FAIL** means that attack still works.

If a red badge appears at the top of the page

Your saved file could not be loaded, usually a Python mistake. The badge shows the error text. Until you fix it, save, and press **Reset Scenario**, the page runs a built-in stand-in instead of your code.

4 Reading a FAIL

Each check line names the attack and says what the attacker did. For example:

> **Replay FAIL - attack succeeded**
> Sends the same captured request again without changing it.

The checks run on their own copy of the car, so they do not appear in the Network log. To see exactly what a check sent, run this in a terminal in the project folder:

```
python3 attack_tests.py --verbose
```

Then follow that request through your `create_request`, `verify_request` and `authorize`, and ask what the car server could have noticed. Change one thing, then run the loop in section 3 again. One check at a time.

5 Where each kind of change goes

Each card shows the real code as it is now and where your change goes. The examples use made-up values like `X-Colour`, so they show the *place*, not the answer.

Add something to a request `create_request`

The headers dictionary. It already has one entry.

```
    headers = {
        "X-User": user,
        "X-Colour": "blue",        # your new header

    }
```

A header is a name and a value. Choose any name; `Host`, `Content-Type` and `Content-Length` are added for you. Whatever you put in arrives on the other side as **text**, so a number comes back as `"123"`, not `123`. The Network log shows your extra headers and the start of the body; tick **show full requests** to see everything the Interceptor sees.

Check something when a request arrives `verify_request`

Read a value, test it, refuse if it is wrong. Put your checks **before** the last line that accepts.

```
    user = wire.header(request, "X-User")
    command = str(wire.json_body(request).get("command", "")).upper()

    if not command:
        return {"ok": False, "reason": "no command"}

    colour = wire.header(request, "X-Colour")    # your new check

    if colour != "blue":

        return {"ok": False, "reason": "wrong colour"}


    return {"ok": True, "user": user, "command": command}
```

Write the reason so that it tells *you* which check said no. It appears in the Network log.

Remember something between requests `either side`

Use `client_state` on the client, `server_state` on the car server.

```
    # count how many requests the car server has seen
    server_state["seen"] = server_state.get("seen", 0) + 1
```

Whatever you store is still there on the next request. **Reset Scenario** empties both dictionaries.

Change the body `both sides`

The body is text. The client makes it; the car server reads it.

```
    body = json.dumps({"command": command})     # in create_request
```

You may turn that text into any other text before sending it. Then, in `verify_request`, turn it back before you read the command. If the body no longer contains a plain `"command"` value, `wire.json_body(request)` cannot read it for you: take `request["body"]`, undo your change, then `json.loads(...)` to get the dictionary back.

The password both sides

The page puts whatever was typed in the Password box into `client_state["password"]`.

How the client sends it, and how the car server checks it, is your decision. `PASSWORDS` at the top of the file is a place for whatever the car server needs to keep. The built-in scenarios keep theirs in `labusers.py`, which you can read.

Decide who may do what authorize

Called only after `verify_request` said yes.

```
def authorize(user, action, server_state):
    return True                    # right now, everyone may do everything
```

`action` is `"move"` for the arrows and `"admin_reset"` for Workshop reset. `USER_ACTIONS` at the top of the file is one way to write the rule.

Take over the maintenance door optional

The bottom of the file has this as a comment. Copy it out, remove the `#` from each line, fill in the middle.

```
def maintenance_auth(request, server_state):
    guess = str(wire.json_body(request).get("code", ""))
    ...
    return {"ok": True}    # or {"ok": False}
```

If the function does not exist, the car server keeps handling that door itself.

Say what you built PROPERTIES

```
PROPERTIES = {
    "Integrity": False,
    ...
}
```

Set to `True` the ones you believe your design provides. This only colours the badges on the page. The security checks decide for themselves.

6 Mistakes people make

What happened	Why

What happened	Why
Saved the file, nothing changed on the page.	You did not press **Reset Scenario**. The page is still running the old file.
Every request is rejected and the reason is `no result`.	`verify_request` returned nothing. It must return a dictionary with `"ok"` in it.
A check on a number in a header never matches.	Headers arrive as text. Compare with `str(n)`, or convert with `int(...)` first.
Honest presses are rejected too, not just attacks.	`verify_request` checks for something `create_request` never sent. Both sides must agree.
A value you stored is missing.	You stored it in `client_state` and looked in `server_state`, or the other way round. Two dictionaries, two sides.
It worked until Reset Scenario.	Your design relies on something stored in a dictionary from before. Reset empties both. Decide what happens on a fresh start.